COMP4321 Web Search Engine Project Report

YUE Pu (pyue@connect.ust.hk) Gyungbin JO (gjoaa@connect.ust.hk)

LI Linfeng (lliel@connect.ust.hk)

May 9, 2026

Contents

1 Overall Design of the System

Our web search engine is designed with a modular architecture consisting of three primary components: the Spider (Crawler), the Indexer, and the Search Engine with its Web Interface. The system is implemented in Python 3.8+ and uses SQLite as the persistent storage layer via the `sqlite3` module. The high-level data flow is: crawler fetches pages, indexer stores metadata and postings, and the Flask web app queries the search engine for ranked results.

1.1 Component Interaction

1. **Spider (crawler.py)**: Fetches web pages recursively using a breadth-first search strategy. For each fetched page, it extracts hyperlinks for further crawling and passes the page content to the indexer.

2. **Indexer (indexer.py, stop_stem.py)**: Processes raw HTML content, removes stop words, applies Porter stemming through the NLTK stemmer, and updates the SQLite database with page metadata, word mappings, link relations, and postings.

3. **Search Engine (search_engine.py)**: Implements a TF-IDF-based vector space ranking model with cosine similarity. It supports free-text queries and exact quoted phrases by consulting positional postings stored in SQLite.

4. **Web Interface (app.py, templates/)**: A lightweight Flask application that accepts user queries, passes them to the search engine, and renders ranked results in an HTML page.

2 File Structures Used in the Index Database

We use SQLite (single file: `spider.db`) as our database backend. The schema is kept simple and matches the implementation in `indexer.py`. The database design document is available in `database_design.txt`. At a high level, the key tables are:

2.1 Page Table (`page`)

Stores metadata for each crawled and indexed URL.

- `page_id INTEGER PRIMARY KEY AUTOINCREMENT`: Unique internal ID.
- `url TEXT UNIQUE NOT NULL`: Full URL of the page.
- `title TEXT`: Extracted page title, populated after fetching.
- `last_modified TEXT`: Last-Modified header value, or Date header if missing.
- `size INTEGER`: Page size in bytes.

2.2 Word Table (`word`)

Stores unique stemmed words (tokens).

- `word_id INTEGER PRIMARY KEY AUTOINCREMENT`: Unique ID for the stemmed word.
- `word TEXT UNIQUE NOT NULL`: The stem itself.

2.3 Inverted Index Tables

Two separate tables are created to differentiate between body text and title text, as required.

- `posting_body (word_id, page_id, freq, positions)`: Maps a stemmed word to a page where it appears in the body.
- `posting_title (word_id, page_id, freq, positions)`: Maps a stemmed word to a page where it appears in the title.

Each entry stores the term frequency (`freq`) and a JSON-encoded list of positions (`positions`) to support phrase search.

2.4 Linkage Table (`link`)

Stores parent-child relationships between pages to support navigational display.

- `parent_id INTEGER`: Page ID of the parent page.
- `child_id INTEGER`: Page ID of the child page.
- `PRIMARY KEY (parent_id, child_id)`.

2.5 Keyword Frequency Table (`keyword_freq`)

Stores body-term frequencies for each page so that the web interface and output script can display the most frequent keywords without rescanning postings.

- `page_id INTEGER`: Page ID.
- `word_id INTEGER`: Word ID.
- `freq INTEGER`: Body frequency of the word in that page.
- `PRIMARY KEY (page_id, word_id)`.

3 Algorithms Used

3.1 Breadth-First Search (BFS) Crawling

The spider uses a BFS queue and a visited set. It fetches each page, skips failures, re-indexes only when the last-modified value changes, records links, and commits each crawl batch.

```python
def crawl(self):
    visited = set()
    queue = deque([self.seed_url])

    while queue and len(visited) < self.max_pages:
        url = queue.popleft()
        if url in visited:
            continue
        visited.add(url)

        print(f"[{len(visited)}/{self.max_pages}] Crawling: {url}")
        page_id = self.indexer.get_page_id(url)

        response, soup = self.fetch_page(url)
        if soup is None:
            continue

        last_modified = self.get_last_modified(response)

        if self.should_recrawl(page_id, last_modified):
            title = soup.title.string.strip() if soup.title and soup.title.
    string else url
            body_text = soup.get_text(separator=" ")
            size = len(response.content)

            self.indexer.clear_page_index(page_id)
            self.indexer.add_page_info(page_id, title, last_modified, size)
            self.index_text(page_id, title, is_title=True)
            self.index_text(page_id, body_text, is_title=False)

        child_links = self.extract_links(soup, url)
        for child_url in child_links:
            child_id = self.indexer.get_page_id(child_url)
            self.indexer.add_link(page_id, child_id)
            if child_url not in visited:
                queue.append(child_url)

        self.indexer.commit()

    self.indexer.remove_pages_not_in(visited)
    self.indexer.clean_stubs()
```

Listing 1: BFS Crawling Core Logic

3.2 The Vector Space Model and TF-IDF

We implement a vector space model where both documents and queries are represented as weighted term vectors.

- **Term Frequency (TF)**:

$$tf(t,d) = \frac{f_{t,d}}{\max_{\tau \in d} f_{\tau,d}} \tag{1}$$

where $f_{t,d}$ is the frequency of term t in document d.

- **Inverse Document Frequency (IDF)**:

$$idf(t) = \log_2 \left(\frac{N}{df_t} \right) \tag{2}$$

where N is the total number of documents and df_t is the number of documents containing term t.

- **TF-IDF Weight**:

$$w_{t,d} = tf(t,d) \times idf(t) \tag{3}$$

3.3 Cosine Similarity

Document and query similarity is computed using the cosine of the angle between their weighted vectors.

$$\text{cosine\_sim}(Q, D) = \frac{\sum_{t \in Q} w_{t,Q} \cdot w_{t,D}}{\sqrt{\sum_{t \in Q} w_{t,Q}^2} \cdot \sqrt{\sum_{t \in D} w_{t,D}^2}} \tag{4}$$

3.4 Favoring Title Matches

The search engine merges title postings into the document vector with an extra weight multiplier. In the implementation, title terms are weighted by a factor of 2.0 before cosine similarity is computed. This is handled directly in `search_engine.py`.

3.5 Phrase Search Handling

To support exact phrase queries (e.g., `"hong kong"`), we:

1. Split the phrase into individual stemmed terms.
2. Retrieve the posting lists with positions for each term from the relevant posting table.
3. Perform a positional check to confirm that the terms appear consecutively in the same document.
4. Return only documents where the consecutive condition holds.

3.6 Link Structure and Cyclic Handling

The crawler maintains a `visited` set while crawling and stores each extracted hyperlink as a parent-child record in the `link` table. The search engine and output script read this table to retrieve parent and child links.

4 Installation Procedure

4.1 Prerequisites

- Python 3.8 or higher

- pip (Python package installer)
- Internet access for initial crawling

4.2 Setup and Execution

The project can be set up and run by following the instructions in `readme.txt`:

```
# 1. Clone the repository
git clone <repository-url>
cd COMP4321

# 2. Install Python dependencies
pip install -r requirements.txt

# 3. Run the spider to index pages
python crawler.py

# 4. Start the web interface
python app.py
#    Access the search engine at http://127.0.0.1:5000/
```

Listing 2: Installation and Execution Commands

Note: No extra NLTK data download is required for the current implementation because the project uses Porter stemming only. A simple `Makefile` was not created as the project is purely Python-based and does not require a compilation step.

5 Highlights of Features Beyond the Required Specification

Our team implemented several enhancements to improve robustness, user experience, and search quality:

- **Re-crawling Check**: The spider compares the `Last-Modified` header before re-indexing a known URL. If the page has not changed, it is skipped.
- **Phrase Search**: Exact phrase search is supported using positional postings.
- **Flask Web Interface**: The search UI is implemented with Flask and Jinja2 templates.
- **Keyword Display**: Search results show the most frequent body keywords for each page.
- **Parent and Child Link Display**: Both parent and child links are retrieved and displayed for each search result, providing a full navigational context as required.

6 Testing of Functions Implemented

This section summarizes behaviors that are directly supported by the implementation.

6.1 Spider and Indexer Testing (Phase 1)

- **Test Case 1.1: BFS and Page Limit**: The crawler uses a queue-based BFS traversal and stops when `len(visited)` reaches `max extunderscore pages`.
- **Test Case 1.2: Stop Word & Stemming**: The crawler tokenizes text with a regular expression, removes stop words using `StopStem`, and applies Porter stemming before indexing.
- **Test Case 1.3: Link Extraction**: The crawler extracts only `http` and `https` links, strips fragments, and inserts parent-child pairs into the link table.
- **Test Case 1.4: Re-crawling Robustness**: The crawler compares the current `Last-Modified` or `Date` value with the stored value and re-indexes a page only when the values differ.

6.2 Search Engine and Web Interface Testing

- **Test Case 2.1: Basic Vector Space Query**: The search engine tokenizes a query, removes stop words, stems the remaining tokens, and computes query weights from the stored document statistics.
- **Test Case 2.2: Title Weight Verification**: The search engine reads from both `posting_body` and `posting_title`, then scales title frequencies by `TITLE_WEIGHT` before cosine similarity is computed.
- **Test Case 2.3: Exact Phrase Search**: For quoted phrases, the search engine checks positional lists in both body and title postings and only keeps documents where each term appears consecutively.
- **Test Case 2.4: Web Interface Formatting**: The Flask app passes `results`, `query`, and `error` to `index.html`; the template renders score, title, URL, last modified date, size, keywords, parent links, and child links when present.
- **Test Case 2.5: Max Results**: The search engine truncates the ranked result list to the first 50 documents.

7 Conclusion

7.1 Strengths of the System

- **Simplicity and Portability**: Using Python and SQLite means the entire engine is self-contained in a single directory with no external database server to configure. Running it requires only Python and a few pip-installed libraries.
- **Correctness**: The system implements BFS crawling, stop-word removal, stemming, phrase search, title weighting, and re-crawl checks.

- **Extensibility**: The modular design (crawler, indexer, search engine as separate modules) makes it easy to extend or replace parts. The SQLite schema is normalized, allowing for complex queries.

7.2 Weaknesses and Limitations

- **Scalability**: SQLite is a lightweight database and not designed for high-concurrency or internet-scale data. Performance would degrade with millions of pages. A production system would require a distributed index (e.g., Elasticsearch or Apache Lucene) and a dedicated database server (e.g., PostgreSQL).
- **Crawling Speed**: The single-threaded BFS crawler is slow and network-bound. Parallel fetching using asynchronous libraries (`aiohttp`) or a thread pool would be necessary for large-scale crawling.
- **Ranking Algorithm**: Our ranking relies purely on TF-IDF and a simple title boost. We did not implement advanced features like PageRank or BM25, which would significantly improve result relevance.

7.3 Potential Improvements and Interesting Features

If we were to re-implement the system or extend it, we would consider:

- **PageRank or BM25**: Incorporate a stronger ranking model to improve relevance.
- **Relevance Feedback**: Implement a similar-pages feature using the top keywords from a chosen result.
- **Better Query Handling**: Add query expansion and spelling suggestions.
- **User Experience**: Develop a more interactive UI with query history and richer filters.

7.4 Lessons Learned

The most significant architectural decision was the choice of SQLite. While it served us well for rapid development and met the project requirements, it also makes clear the tradeoff between simplicity and scalability. Building a search engine from scratch also highlighted the importance of preprocessing, inverted index design, and careful handling of link structure and ranking.

8 Contribution

The project contributions were relatively balanced, with the following main areas of responsibility:

- **YUE Pu**: Retrieval function and search-related logic.
- **Gyungbin JO**: Spider and web interface.
- **LI Linfeng**: Test program and document writing.